Evet, bu sizi doğrudan kernel (çekirdek) programlamaya veya işletim sistemi geliştirmeye götürür.

Bilgisayar mimarisinde bellek yönetimi (Memory Management), donanım ile yazılım arasındaki en kritik köprüdür. Modern AMD işlemcilerde (x86_64 mimarisi) belleği manuel olarak yönetmek ve yazdığınız kodun hiçbir aracı olmadan doğrudan **binary (ikili makine kodu)** olarak çalışmasını sağlamak, geleneksel uygulama programlamasının tamamen dışındadır.

Bunun nedenlerini ve bu sürecin sizi hangi alanlara götüreceğini aşağıda görebilirsiniz:

🌟 Sizi Bekleyen Seviyeler

- **İşletim Sistemi (OS) Geliştirme (Bare-Metal Programming):** Eğer mevcut bir işletim sistemi (Windows, Linux vb.) olmadan, bilgisayar açılır açılmaz kodunuzun doğrudan AMD işlemci üzerinde çalışmasını istiyorsanız, bir "Bare-Metal" programcı olursunuz. Kendi bootloader'ınızı (önyükleyici) yazmalı ve işlemciyi **Real Mode**'dan **Protected Mode** ve ardından **Long Mode** (64-bit) seviyesine geçirmelisiniz.
- **Kernel (Çekirdek) Sürücüsü Yazmak:** Eğer mevcut bir işletim sisteminin içinde, ancak donanıma tam erişimli bir bellek yönetimi yapmak istiyorsanız bir Linux Kernel Modülü (LKM) veya Windows Sürücüsü (Driver) yazmalısınız. Bu da kernel programlamanın ta kendisidir.

⚙️ AMD İşlemcide Bellek Yönetimi İçin Neler Yapmalısınız?

Modern AMD işlemcilerde belleği manuel yönetmek için şu donanımsal yapıları doğrudan manipüle etmeniz gerekir:

- **Paging (Sayfalama) Tabloları:** İşlemcinin fiziksel bellek adresleri ile sanal bellek adreslerini eşlemesi için CR3 kontrol yazmacına (register) kendi oluşturduğunuz sayfa tablolarının (PML4, PDPT, PD, PT) adresini yazmalısınız.
- **GDT (Global Descriptor Table):** Bellek segmentasyonunu ve koruma seviyelerini (Ring 0 - Kernel, Ring 3 - User) belirlemek için GDT yapısını assembly ile kurmalısınız.
- **MMIO (Memory-Mapped I/O):** AMD sistemlerdeki diğer donanımlara veya işlemcinin kendi entegre bileşenlerine erişmek için belirli fiziksel bellek adreslerini manuel olarak haritalandırmalısınız.

🛠️ Hangi Dilleri ve Araçları Kullanmalısınız?

Bu seviyede yüksek seviyeli dillerin (Python, C# vb.) çalışma zamanı (runtime) motorları olmadığı için şu dilleri kullanmak zorundasınız:

- **Assembly (x86_64 NASM/GAS):** İşlemci yazmaçlarını (CR0, CR3, CR4) kontrol etmek ve sayfalama modlarını aktif etmek için şarttır.
- **Freestanding C / Rust:** Hiçbir standart kütüphaneye (stdio.h veya std:: gibi işletim sistemi bağımlı yapılar) ihtiyaç duymadan doğrudan binary üretebilen modda kod yazmalısınız.

Eğer bu yolculuğa başlamak istiyorsanız, rehber niteliğindeki **"OSDev Wiki"** platformunu inceleyebilir veya Rust diliyle yazılmış popüler **"Writing an OS in Rust" (Philipp Oppermann)** serisine göz atabilirsiniz.

Bu süreçte **mevcut bir işletim sistemi için kernel modülü** mü yazmak istiyorsunuz, yoksa bilgisayarı doğrudan kendi yazdığınız kodla başlatacak **sıfırdan bir sistem** mi geliştirmek istiyorsunuz? Sizi doğru kaynaklara yönlendirmem için projenizin amacını paylaşabilirsiniz.